SEMESTER PROJECT REPORT

CACHE ME IF U CAN

Design and Verification of a Direct-Mapped Cache Controller

using Verilog HDL

Course

Modern Computer Architecture

Project Team

Agrima Ojha	2401010047
Khyati Batra	2401010229
Khushi Jain	2401010227
Suhani Gupta	2401010463
Akshat Agrawal	2401010054

Project: Cache Me If U Can

Verilog RTL Design | Functional Simulation | Synthesis Analysis

Abstract

This paper presents the design and simulation-based verification of a 32-bit, direct-mapped cache controller implemented in Verilog. The cache has a data capacity of 1 KB, is organized as 64 lines, and stores one 16-byte block per line. Each block contains four 32-bit words. The design is divided into an address decoder, cache storage, main-memory model, memory interface, and a five-state finite-state-machine (FSM) controller. On a cache miss, the memory interface reads four consecutive words and supplies a 128-bit block for cache refill. The integrated testbench exercises sequential, repeated, random, and conflict access traces. The project report records 42 accesses, 19 hits, 23 misses, and zero data-verification errors, corresponding to a 45.24% hit rate and a reported average memory access time (AMAT) of 4.83 cycles. Yosys synthesis completed successfully and reported 116,340 generic mapped cells; target-specific timing and maximum frequency were not measured.

Project at a glance

Cache capacity	1 KB	**Cache lines**	64
Block size	16 bytes	**Words per block**	4
Address width	32 bits	**Mapping**	Direct mapped

Keywords: Direct-mapped cache, cache controller, Verilog, finite-state machine, cache hit, cache miss, block refill, memory hierarchy.

Contents

Abstract i

1 Introduction 1

2 Cache Organization 1

2.1 Configuration 1

2.2 Address Decomposition 1

3 System Architecture 2

4 Rtl Implementation 3

4.1 Address Decoder 3

4.2 Cache Storage and Hit/Miss Detection 3

4.3 Main Memory and Block Refill 3

4.4 Cache Controller FSM 3

5 Verification Methodology 4

6 Performance Analysis 4

7 Synthesis Results 5

8 Limitations And Future Work 5

9 Conclusion 5

10 References 6

1. Introduction

A cache is a small, fast storage structure placed between a processor and main memory. It reduces the average time required to access frequently used data by retaining recently fetched memory blocks. A direct-mapped cache assigns each main-memory block to exactly one cache line. This organization is straightforward to implement, although different blocks that map to the same line can evict one another. [1]

The objective of this project is to implement and verify a direct-mapped cache controller using Verilog HDL. The design accepts a CPU read request, decodes its address, checks the selected cache line, initiates a four-word refill on a miss, and returns the requested 32-bit word through a response handshake. The repository includes RTL modules, module-level testbenches, access traces, documentation, simulation results, and a Yosys synthesis log.

2. Cache Organization

2.1 Configuration

The implementation uses a 32-bit byte-addressed interface and the configuration summarized in Table I. The cache data array contains 64 entries, each 128 bits wide. The resulting data capacity is 64×16 bytes = 1,024 bytes (1 KB).

Table 1: Cache configuration

Parameter	Implemented value
Address width	32 bits
Cache data capacity	1 KB (1,024 bytes)
Number of cache lines	64
Block size	16 bytes
Word size	32 bits
Words per block	4
Mapping	Direct mapped

2.2 Address Decomposition

The address decoder uses bit slicing to produce the tag, line index, block offset, and word index. Bits [31:10] form the 22-bit tag; bits [9:4] form the 6-bit index; and bits [3:0] form the 4-bit byte offset. The word index is taken from bits [3:2], selecting one of the four words in a block. Bits [1:0] represent the byte position within the selected 32-bit word.

Table 2: Address field allocation

Field	Address bits	Width
Tag	[31:10]	22 bits
Index	[9:4]	6 bits
Block offset	[3:0]	4 bits
Word index	[3:2]	2 bits

For example, address 0x00000024 produces tag 0, index 2, offset 4, and word index 1. Address 0x00000400 produces tag 1, index 0, offset 0, and word index 0. Addresses 0x00000020, 0x00000420, and 0x00000820 all select index 2 but have different tags, making them a conflict-miss sequence.

3. System Architecture

The integrated design is organized around the cache_top module. The address_decoder extracts the address fields. The cache_storage module contains the data, tag, and valid-bit arrays and performs tag comparison and word selection. The cache_controller coordinates request acceptance, lookup, miss handling, refill waiting, and response delivery. The memory_interface sequences four main-memory reads and assembles the refill block. The main_memory module models 1,024 words of 32-bit storage, or 4 KiB of simulated main memory.

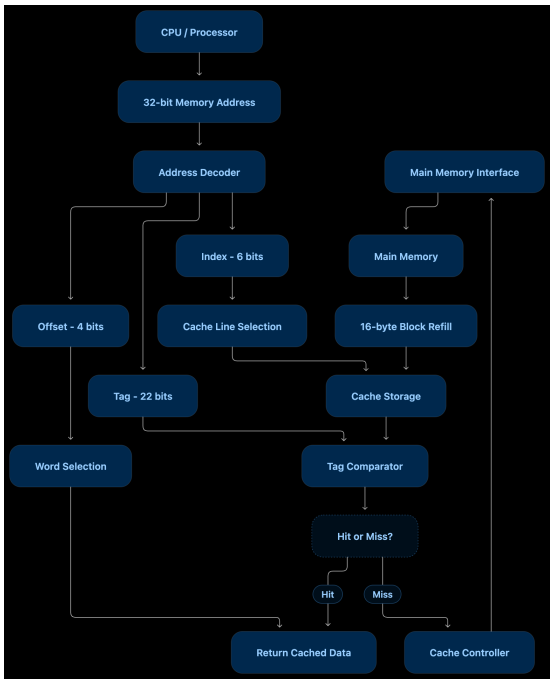


Figure 1: Block-level architecture of the direct-mapped cache controller.

The main-memory model provides a combinational read path and a clocked write path. Its write interface is used by the testbenches to initialize memory contents. The cache itself is read-only from the CPU-facing interface in the submitted implementation; no CPU store/write-through or write-back policy is implemented.

4. Rtl Implementation

4.1 Address Decoder

The address_decoder module is combinational. It maps address[31:10] to tag, address[9:4] to index, address[3:0] to offset, and address[3:2] to word_index. The module does not contain sequential state. [2]

4.2 Cache Storage and Hit/Miss Detection

The cache_storage module declares data_array[0:63] as 128-bit entries, tag_array[0:63] as 22-bit entries, and valid_array[0:63] as one-bit entries. A lookup is a hit only when lookup_valid is asserted, the selected line is valid, and the stored tag equals the requested tag. A valid lookup that does not satisfy these conditions is reported as a miss. During a hit, word_index selects one of the four 32-bit slices of the stored block. Reset clears the valid bits. When fill_valid is asserted, the selected line receives the incoming block and tag and its valid bit is set.

4.3 Main Memory and Block Refill

The main_memory model contains 1,024 32-bit words. The memory_interface aligns a miss address to a 16-byte block boundary by clearing its low four bits. It then sequences through READ0, READ1, READ2, and READ3 to capture the four consecutive words at offsets 0, 4, 8, and 12 bytes. The words are packed into a 128-bit block in the order {word3, word2, word1, word0}. In the FILL state, fill_valid is asserted and the cache storage captures the block, tag, and index.

4.4 Cache Controller FSM

The cache_controller uses five states: IDLE (000), LOOKUP (001), MISS (010), REFILL (011), and RESPONSE (100). In IDLE, request_ready is asserted and a valid request address is saved. LOOKUP evaluates the cache hit and miss signals. A hit captures the requested data and moves to RESPONSE; a miss moves to MISS. MISS asserts a one-cycle memory_request pulse and advances to REFILL. REFILL waits for fill_valid and then returns to LOOKUP so the refilled line can be read. RESPONSE holds response_valid and response_data until response_ready is asserted, after which the FSM returns to IDLE.

Table 3: Controller FSM states

State	Purpose / transition
IDLE	Accept and latch a CPU request
LOOKUP	Check tag/valid bit; hit → RESPONSE, miss → MISS
MISS	Issue memory request
REFILL	Wait for block fill, then retry lookup
RESPONSE	Hold data until response_ready

5. Verification Methodology

The repository contains module-level testbenches for address decoding, cache storage, memory refill, and controller operation, along with an integrated cache testbench. The address decoder testbench checks four representative addresses. The cache-storage testbench checks an empty-line miss, refill, hit detection, selection of all four words, tag mismatch, invalid-line miss, and disabled lookup. The memory testbench initializes four adjacent words, requests a block, and checks that all four words are available after refill. The controller testbench checks a first miss/refill response, a repeated hit, and accesses to other words in the same block.

The integrated testbench initializes the simulated main memory and executes four address traces: sequential, repeated, random, and conflict. For every access, the testbench compares the returned data with an expected-memory array and records hit/miss counts and latency. The reported simulation summary in the repository is reproduced in Table IV.

Table 4: Reported integrated simulation results

Trace	Accesses	Hits	Misses	Errors
Sequential	12	9	3	0
Repeated	9	8	1	0
Random	12	2	10	0
Conflict	9	0	9	0
Total	42	19	23	0

6. Performance Analysis

The reported hit rate is calculated as hits divided by total accesses: 19/42 = 45.24%. The miss rate is 23/42 = 54.76%. The repository records an average hit latency of 1 cycle and an average miss latency of 8 cycles, giving a miss penalty of 7 cycles. Using AMAT = hit time + (miss rate × miss penalty), the reported value is $1 + (23/42 \times 7) = 4.83$ cycles.

The sequential trace benefits from spatial locality because one refill supplies four consecutive words. The repeated trace benefits from temporal locality after its first block is loaded. The random trace has less reuse across the selected addresses. In the conflict trace, three addresses map to the same cache index with different tags; each new block replaces the previous one, so the reported trace has nine misses and no hits.

These figures are the results recorded in reports/performance_analysis.md and reports/simulation_results.txt in the submitted repository. They are presented as repository-reported simulation results, not as independently re-measured values in this document.

7. Synthesis Results

The project repository records a successful Yosys synthesis of cache_top using Yosys version 0.70+post. The synthesis report lists 116,340 generic mapped cells in the design hierarchy. The module-level counts recorded are 93 cells for cache_controller, 25,487 for cache_storage, 273 for memory_interface, and 90,487 for main_memory. The address decoder is represented primarily by wiring and does not add mapped logic cells in the report. [3]

The main-memory model accounts for most of the generic cell count because it describes 1,024 words of 32-bit storage. The cache storage also contributes a substantial count because it includes 64×128-bit data entries and associated tag and valid-bit storage. These generic mapped-cell counts must not be interpreted as FPGA LUT utilization, physical silicon area, or a device-specific resource estimate. The report explicitly states that maximum operating frequency was not measured; target technology, timing libraries, clock constraints, and static timing analysis would be needed for a meaningful frequency result.

8. Limitations And Future Work

The submitted CPU-facing design implements read requests and a simulated main-memory initialization/write port for test purposes. It does not implement processor store instructions, dirty bits, write-back, or write-through behavior. The memory model uses a simple combinational read path and should be treated as a functional simulation model rather than a timing-accurate external memory device. The synthesis result is technology independent and no maximum-frequency result is available.

Future extensions may include CPU write support with a clearly specified write policy, parameterized cache geometry, a registered or latency-controlled memory model, more extensive randomized verification, assertions for request/response protocol properties, and target-device implementation followed by static timing analysis.

9. Conclusion

A modular direct-mapped cache controller has been implemented in Verilog using a 1 KB cache with 64 lines and 16-byte blocks. The design combines address decoding, valid-bit and tag-based hit/miss detection, four-word block refill, and a five-state controller FSM. The repository records successful integrated simulation of 42 memory accesses with zero data-verification errors, a 45.24% hit rate, and an AMAT of 4.83 cycles. Yosys synthesis completed successfully and reported 116,340 generic mapped cells. The project demonstrates the operation of direct mapping, spatial and temporal locality, conflict misses, and the control sequence required to service a cache miss.

[1] D. A. Patterson and J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition, 2nd ed. Cambridge, MA, USA: Morgan Kaufmann, 2021.

[2] IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005, 2006.

[3] Yosys Open SYnthesis Suite, “Yosys Documentation.” [Online]. Available: https://yosyshq.readthedocs.io/.

[4] "Cache-me-if-u-can," project source code, RTL modules, testbenches, traces, and reports, GitHub repository: https://github.com/Khushi-jain05/Cache-me-if-u-can.

10. References

[1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, RISC-V Edition, 2nd ed. Cambridge, MA, USA: Morgan Kaufmann, 2021.

[2] IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005, 2006.

[3] Yosys Open SYnthesis Suite, "Yosys Documentation." [Online]. Available: https://yosyshq.readthedocs.io/.

[4] "Cache-me-if-u-can," project source code, RTL modules, testbenches, traces, and reports, GitHub repository: https://github.com/Khushi-jain05/Cache-me-if-u-can.